Problem Statement

Task: signal $X \in \mathbb{R}^{C \times T}$ → Action Prediction

Dataset:

Physionet => 109 subjects, 64 channels

1 subject => 14 runs => same session (real/imagery)

2 runs -> eyes open / closed (baseline)

4 runs -> Task 1 (real => left vs right fist)
Task 2 (imagined => left vs right fist)
Task 3 (real => both fists vs both feet)
Task 4 (imagined => both fists vs both feet)

x2 more times => Total 14 runs.

So total 6 real vs 6 imagery runs.

1 run = 2min 5s => 125s

each run => 4s interval between actions

4s rest (T0), 4s action (T1/T2)

Total 15 actions (7-8 T1/T2)

Sampling rate => 160 Hz → 125 s => 20000 samples

$C \in \{1, 2, 3 \ldots 64\}$

$T \in \{1, 2, \ldots 20000\}$

BCIIV-2a : 9 subjects, 22 EEG + 3 EOG

1 subject : 2 sessions (T & E) => cross session

Actions : Left hand, right hand, both feet, Tongue

each session : 288 trials => 72 x 4 actions

each trial : 7-8 s => 2s cue, 4s action, rest

Sampling frequency = 250 Hz.

Total time / session = 288 x (7-8) ≈ 2000-2400 s

No. of samples : 250 x (2000-2400) = 500000 - 600000 samples

Dataset representation

$X_{i,a} \in \mathbb{R}^{C \times T}$; $i = \{1, 2, 3 \ldots 109$ subjects$\}$, $a \in$ Action

T' = epoch time (4s in Physionet, 3s/4s after cue)

Preprocessing

RIEMANNIAN MANIFOLD.

Distance => geodesic distance

$X_{i,a} \in \mathbb{R}^{C \times T}$; $C = \{1, 2 \ldots 64\}$, $T = \{1, 2 \ldots 20000\}$

$C_{i,a} = \frac{1}{C-1} X_{i,a} X_{i,a}^T \in SPD(C) \Rightarrow C \times C$ matrices

AIRM geodesic distance

$d(C_1, C_2) = \| \log (C_1^{-1/2} C_2 C_1^{-1/2}) \|_F$

symmetric: $d(C_1, C_2) = d(C_2, C_1)$ — (i)

Mean (N_T : No of trials)

Riemannian Mean of subject i : $C_{R,ref,i} = \underset{C}{\operatorname{argmin}} \sum_{e,a} d_R^2(C, C_{i,a,e})$

Euclidean Mean of i : $C_{E,ref,i} = \frac{1}{N_T} \sum_{e,a} C_{i,a,e}$ — (ii)

Log-Euclidean Mean of i : $C_{LE,ref,i} = \frac{1}{N_T} \sum_{e,a} \log C_{i,a,e}$ (iii)

Whitening

$C_{R,ref,i}$ is Riemannian mean of subject i.

Consider a matrix $C_{i,a,e}$ of subject i

$$C'_{i,a,e} = C_{R,ref,i}^{-1/2} C_{i,a,e} C_{R,ref,i}^{-1/2} \quad \text{— (iv)}$$

=> We do this because, each subject i has $C_{R,ref,i}$ reference that is used to identify the subject. Therefore if Cref of all subjects are I, then subjects are generalized & no cues for identifying subject

$$X'_{i,a,e} = C_{R,ref,i}^{-1/2} X_{i,a,e} \quad \text{— (v)}$$

Here X' is normalized, $\sim x/\sigma$.

So μ/σ => deviation from I where all x_i/σ => same μ. So this can be used specifically for classification.

RIEMANNIAN METHODS

Tangent Space Logistic Regression (CSP)

1. $X_{i,a,e}, a \in A$, $A \in [0, 1, 2, 3$ => Physionet$]$, $e \in [1, 2 \ldots N_T]$

2. Compute Covariance $C_{i,a,e}$ of each $X_{i,a}$

$$C_{i,a,e} = \frac{1}{C-1} X_{i,a,e} X_{i,a,e}^T$$

3. Compute per subject Cref.

Riemannian Mean : $C_{R,ref,i} = \underset{C}{\operatorname{argmin}} \sum_e d_R^2(C, C_{ie})$

Euclidean Mean : $C_{E,ref,i} = \frac{1}{N_T} \sum_e C_{ie}$

Log-Euclidean Mean : $C_{LE,ref,i} = \frac{1}{N_T} \sum_e \log C_{ie}$

4. Align the Covariances to the per subject ref such that C_{ref} of updated covariances per subject = I

We can use $C_{ref,i} = C_{R,ref,i}$ $C_{E,ref,i}$, $C_{LE,ref,i}$ depending on type of Alignment.

$$C'_{i,a,e} = C_{ref,i}^{-1/2} C_{i,a,e} C_{ref,i}^{-1/2}$$

5. Compute $C_{ref}^{global} = \underset{C}{\operatorname{argmin}} \sum_{i,a,e} \| \log C^{-1/2} C'_{i,a,e} C^{-1/2} \|_F$

6. Project C' onto tangent space at $C_{g,ref}$

$$T_{i,a,e} = C_{g,ref}^{1/2} \log (C_{g,ref}^{-1/2} C'_{i,a,e} C_{g,ref}^{-1/2}) C_{g,ref}^{1/2}$$

7. Perform Multinomial Logistic Regression -

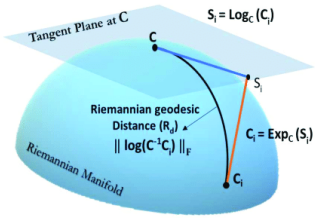
8. $x_{i,a,e} = vec_D(T_{i,a,e})$

9. $x_{(i-1)*N+j} = \begin{cases} T_{ij} & ; i = j \\ \sqrt{2} T_{ij} & ; i \neq j \end{cases}$

10. $z_k = w_k^T x_i + b_k$.

$\hat{y}_k = \sigma(z_k) = \frac{e^{z_k}}{\sum_k e^{z_k}}$ $k = \{0, 1, 2, 3, 4\}$

$y_k = \{0, 1, 2, 3\}$.



Minimum Distance to Mean (MDM)

Algorithm

1. $X_{i,a,e}$; $i \in \{1,2 \dots 109$ subjects => Physionet$\}$
   $a \in \{0,1,2,3$ => actions$\}$
   $e \in \{1,2,3 \dots N_T\}$

2. Compute Covariances $C_{i,a,e}$

$$C_{i,a,e} = \frac{1}{C-1} X_{i,a,e} X_{i,a,e}^T$$

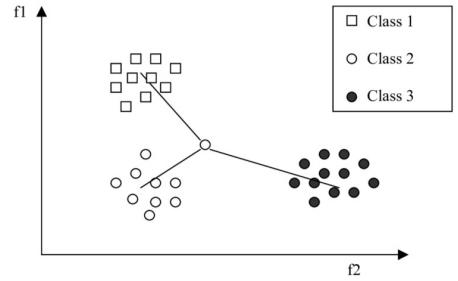
3. Compute C_{ref} (global mean) for each class k $\{0,1,2,3\}$ => 4 actions

* Euclidean Mean$(k) = \frac{1}{N} \sum_{i,e} C_{i,e,a}\big|_{a=k}$
* Riemannian Mean(k) (MDRM) $= \underset{C}{\text{argmin}} \sum_{i,e} d_R^2(C, C_{i,e,a})\big|_{a=k}$
* Log-Euclidean Mean$(k) = \frac{1}{N} \sum_{i,e} \log C_{i,e,a}\big|_{a=k}$

4. For covariance $C_{i,e,a}$, for $k \in \{0,1,2,3\}$

$$D_k(C_{i,e,a}, C_{ref,k}) = \| \log(C_{i,e,a}^{-1/2} C_{ref,k} C_{i,e,a}^{-1/2}) \|_F$$

5. $\hat{y}_k = \underset{k}{\text{argmin}}\ D_k(C_{i,e,a}, C_{ref,k})$



Common Spatial Patterns (CSP)

Similar to PCA, maximizes variance, but CSP maximizes variance of 1 class while reducing variance of other classes. Finds all such vectors.

Algorithm

1. Consider k classes of signals $X_{k1}, X_{k2} \dots X_{kn}$
2. Compute Covariances of each signal

$$C_{ki} = \frac{X_{ki} X_{ki}^T}{\text{trace}(X_{ki} X_{ki}^T)}$$ => trace normalization

3. Compute average of covariances per class

$$C_k = \frac{1}{N_k} \sum_i C_{ki}$$

4. Now we have C_1, C_2, C_3, C_4 for 4 classes
5. So to maximize variance, maximize $J(w)$

$$J(w) = \frac{w^T C_k w}{w^T C_{-k} w} \propto \frac{w^T C_k w}{w^T C_t w}$$

where $C_k = \frac{1}{N_k} \sum_{j \neq k} C_j$, $C_t = \sum_j C_j$

6. Find eigenvectors of $C_k w = \lambda (C_t) w$ (vi)
7. Whiten => $S_k = C_t^{-1/2} C_k C_t^{-1/2}$, Solve $S_k w = \lambda w$
8. Take top m & bottom m eigenvectors (sort λ)

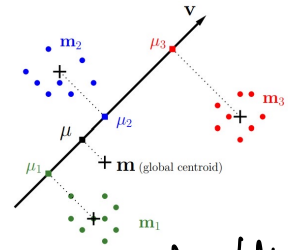
$$W_{MCSP} = [w_1, w_2 \dots \dots w_K]$$
$$X'_{ki} = W_{MCSP}^T X_{ki}$$

9. Use LDA or Multinomial Logistic regression
10. Concatenate X'^T_{ki} for all i & $k \in \{0,1,2,3\}$
11. Logistic Regression $z_k = w_k x'_{ki} + b_k$.

$$\hat{y}_k = \sigma(z_k) = \frac{e^{z_k}}{\sum e^{z_k}}$$ logits.

Linear Discriminant Analysis (LDA)

μ_j => projection
m_j => mean



LDA is supervised method to find vectors/directions that maximizes the variance across classes & minimizes the variance within the class. So it technically maximizes between-class variance & within-class variance.

$$J(V) = \frac{\sum (\mu_j - \bar{\mu})^2}{\sum s_j^2} \Rightarrow \frac{V^T S_b V}{V^T S_w V}$$

μ_j, $\bar{\mu}$, s_j are all variance across points m_j, m, x_j projected on vector V.

$\mu_j = V^T m_j$ & $\bar{\mu} = V^T m$.

$S_b = \sum_j (m_j - m)(m_j - m)^T$ => between class

$S_w = \sum_j \sum_{x \in c_j} (x - m_j)(x - m_j)^T$ => within class

Theorem: Suppose S_w is non-singular, then solution to maximise $J(V)$ is given by highest eigenvectors of $S_w^{-1} S_b$ => $S_w^{-1} S_b V = \lambda V$

1. Compute S_w & S_b for the points X.
2. Solve the generalized eigenvalue problem $S_b V = \lambda S_w V$
   top $K \leq C-1$ eigenvectors $V_K = [v_1, v_2 \dots v_K]$
3. Project X onto them $Z_{1 \times K} = X_{1 \times d} \cdot V_{d \times K}$ & classify
   Z is low-dimensional LDA feature vector.
4. $\hat{y}_c = \underset{c}{\text{argmax}} \left(z^T \Sigma^{-1} \mu - \frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c + \log P(c) \right)$ (vii)

Assuming each class c follows Gaussian with variance Σ $= \mathcal{N}(z; \mu_c, \Sigma)$

Riemann Procrustes Analysis (RPA)

This process generally has 3 steps
-> re-center => scale => rotate.

* Limitation : This is a transfer learning method. Can't use for cross subject as we don't have labeled test data.

Source (S) : train
Test (T) : test

* re-center => Riemannian Alignment.

$C' = C_{ref}^{-1/2} C C_{ref}^{-1/2}$ => $\boxed{C'_{ref} = I}$

* scale: we need to find the dispersions between the source (train) & target (test)

train dispersions : $\sigma_S^2 = \frac{1}{|S|} \sum_{C' \in S} d^2(C', I)$ => $\log C'_S$

test dispersions : $\sigma_T^2 = \frac{1}{|T|} \sum_{C' \in T} d^2(C', I)$ => $\log C'_T$

$\rho = \sqrt{\sigma_S^2 / \sigma_T^2}$ => stretch factor
↳ scalar => euclidean

$Z = \log C'$, $Z' = \rho Z$, $\tilde{C}' = \log Z'$

So $\tilde{C}'$ => covariance with Matched dispersion

* rotate : $\tilde{C}_{ref,S,k} = \underset{C}{\text{argmin}} \sum_{C_S \in S, y=k} d^2(C, C_{S,k})$

$\tilde{C}_{ref,T,k} = \underset{C}{\text{argmin}} \sum_{C_t \in T, y=k} d^2(C, C_{t,k})$

At tangent space, $\tilde{C}_{ref,T,k}$ is to be rotated (per action) aligned to $\tilde{C}_{ref,S,k}$. So the phase direction is same

$T_{T,k} = \log(\tilde{C}_{ref,T,k})$ $T_S = \log(\tilde{C}_{ref,S,k})$

$U = \underset{U}{\text{minimize}} \sum_k \| U^T T_{T,k} U - T_{S,k} \|_F^2$. This update is a type of alignment, After this we can use TSLR or LDA for classification

Observations

Model	X	$C_{ref}^{-1/2}(X - X_{ref})$
CSP + LDA	42.13 ± 15.84 %	45.60 ± 15.81 %

Ablation Study

	ALIGNMENT.			
MODEL	NA	EA	RA	LEA
MDM	39.00 ± 12.18 %	50.81 ± 16.81 %	52.43 ± 15.66 %	52.62 ± 15.52 %
TSLR	33.72 ± 6.80 %	49.50 ± 12.39 %	50.58 ± 12.68 %	50.12 ± 12.92 %
TSA + LDA	36.11 ± 8.79 %	52.93 ± 15.06 %	58.51 ± 15.29 %	58.86 ± 15.60 %
CSP + LDA	42.13 ± 15.84 %	42.13 ± 15.84 %	"	"
CSP + LDA whitening ($C_{ref}^{-1/2}X$)	"	41.67 ± 15.64 %	42.21 ± 15.57 %	41.82 ± 15.70 %
CSP + LDA whitening + z score norm ($C_{ref}^{-1/2}(X - X_{ref})$)	45.60 ± 15.81 %	46.18 ± 15.91 %	45.60 ± 15.81 % (Xref not possible)	46.49 ± 15.22 %

=> Insight: Given a Covariance - C
After eigendecomposition $C = U \Lambda U^T$ where
U is an orthogonal matrix ($U^T U = I$) and
Λ is a diagonal matrix with diagonals being eigenvalues. [$|C - \lambda I| = 0$]

$Cu = \lambda u$ where $[u_1^T, u_2^T \dots]$ compose U [eigenvector] & $\lambda_1, \lambda_2 \dots$ compose diagonal elements of Λ

Covariance => ellipsoid, eigenvectors => axes & eigenvalues are the magnitude of the axes.

=> From RPA we can realize that eigenvectors => actions & eigenvalues => Subject. But not necessarily true

Other Models based on Loss functions.

Models built on CORAL loss, MEKT, MAML & DeepCORAL all are not true zero shot., DANN can be used in true zero shot to penalize subject variance but needs subject label for accuracy improvement

=> Needs test data => Not true zero shot

CORAL => Domain Adaptation (Not DG)

$\mathcal{L}_{CORAL} = \frac{1}{4d^2} \| C_S - C_T \|^2$ (48 - 52 %.)

MEKT => Manifold Embedded Knowledge Transfer (54 ~ 56 %)

Learn a projection matrix W that

$\min_W \alpha \| W^T X_S - W^T X_S L_S \|_F^2 + \beta \| W^T X_T - W^T X_T L_T \|_F + \gamma \, MMD(W^T X_S, W^T X_T)$

- preserves source structure (X_S).
- preserves target manifold structure (X_T)
- minimizes domain discrepancy (MMD)
  => maximum mean discrepancy.

(Needs Target data => does not work instantly on unseen data)

MAML => Model Agnostic Meta Learning (45 %)

Each subject is a task.

Inner loop : simulates learning a subject (source)

$\theta' = \theta - \alpha \nabla_\theta \mathcal{L}_S(\theta)$

Outer loop : Learn parameters to generalize target

$\theta \leftarrow \theta - \beta \nabla_\theta \mathcal{L}_T(\theta)$

PRE-PROCESSING ALGORITHMS.

DCR : Discriminative Class representative Pre-Aligner

1. Input $C_{i,a,e}$
2. Riemann Alignment (RA): Compute AIRM mean & whiten each covariance per subject

$$C'_{i,a,e} = X_{ref}^{-1/2} C_{i,a,e} C_{ref}^{-1/2}$$

3. Compute log map $L_{i,a,e} = \log(C C_{i,a,e})$
4. Compute class mean M_k & global mean M.
5. Initialize rotation & dispersion :
   - Set $A = 0$, compute $R = \exp(A - A^T)$
   - Initialize λ (scaling factor).
6. Optimization loop:
   a. Update rotation : $R = \exp(A - A^T)$
   b. Rotate all log covariances : $L_{R,i,a,e} = R^T L_{i,a,e} R$
   c. Compute discriminative energies.

$$B(R) = \sum_k N_k \| \text{diag}(R^T (M_k - M) R) \|_F^2$$

$$W(R) = \sum_k N_k \| \text{diag}(L_{R,i,a,e} - R^T M_k R) \|_F^2$$

   d. Compute centering penalty (Identity scale)

$$\text{center}(R) = \| \text{offdiag}(L_{R,i,a,e}) \|_F^2$$

   f. Loss fn :

$$L(R, \lambda) = \gamma \frac{W(R)}{B(R) + \epsilon} + \alpha(\lambda - 1)^2 + \frac{1}{N} \|R - I\|_F^2 + \gamma_c \, \text{center}(R)$$

   g. Take gradient step on A & λ.
7. Extract & Save optimized rotation R^*
8. Apply transform
   (i) Rotate using R^* : $L_{R^*,i,a,e} = R^{*T} L_{R,i,a,e} R^*$
   (ii) exp relative to I : $C'_{i,a,e} = \exp_I(L_{R^*,i,a,e})$
9. Output : $C'_{i,a,e}$.

RA + TSLR : 52.89	DCR + TSLR : 53.01 %
RA + MDM : 52.43 %	DCR + MDM : 50.46 %
RA + TSA-LDA : 53.51 %	DCR + TSA-LDA : 53.91 %

RiFU : Riemann Fisher U-Net Pre-Aligner

Input : $C_{i,a,e} \in \mathbb{R}^{c \times c}$ (SPD), $y_{i,act}$, subject ids

Riemann Alignment : $C'_{i,a,e} = C_{ref}^{-1/2} C_{i,a,e} C_{ref}^{-1/2}$

SPD Linear Layer : (My idea => No published paper)

Weight matrices $W \in \mathbb{R}^{c \times d_{hidden}} / \mathbb{R}^{c \times d_{out}}$.

Residual connections : We need to merge input from previous layer with the current input. Similar to addition. Consider C_1 from previous layer & C_2 from current layer we can merge them before sending input

$$C'_{in} = \text{merge}(C_1, C_2) = \exp_I\left(\frac{1}{2}(\log_I C_1 + \log_I C_2)\right)$$

C_{in}' => SPD matrix.

U-Net [Encoder-Decoder Architecture]

$$\phi^E = W^T C'_{i,a,e} W \; ; \; W \in \mathbb{R}^{c \times d}, \; d < c \text{ (encoder)}$$

$$\phi^D = W^T C'_{enc} W \; ; \; W \in \mathbb{R}^{c \times d}, \; d > c \text{ (decoder)}$$

$$U(C) = \phi^{E(n)}(\text{merge}(\phi^{E(n-1)}(\ldots \phi^{E(1)}(C C)\ldots)$$

$$E(U) = \phi^{D(n)}(\text{merge}(\phi^{D(n-1)}(\ldots \phi^{D(1)}(U))\ldots)$$

$E(U)$ = Symmetric (ϵI => eigenvalues => positive definite)

vectorize $E(U)$ & mean $\text{vec}_1(L), v_1, v_2 \ldots$

Compute mean per action, of action means, per subject

$$W(a) = \frac{1}{N_a} \sum_i (v_{i,a} - m_a)(v_{i,a} - m_a)^T$$

$$B(a) = \frac{1}{N_{am}} \sum_i (m_a - m)(m_a - m)^T$$

$$W(s) = \frac{1}{N_s} \sum_i (v_{i,s} - m_s)(v_{i,s} - m_s)^T$$

$$\text{Recon} = \frac{1}{Z_d} \sum \| z_i - z_i^{in} \|_2^2$$

Training loop :

a. Sample covariances. $C_{i,a,e} \in \mathbb{R}^{c \times c}$
b. Forward pass & obtain $E(U C C_{i,a,e}))$
c. Vectorize all $E(U C C))$
d. Compute global, per action, per subject means.
e. Compute loss terms : $W(a)$, $B(a)$, $W(s)$, Recon.

$$L_{RiFU} = \lambda_{within_{act}} W(a) - \lambda_{between_{act}} B(a) - \lambda_{sub} W(s) + \lambda_{recon} \text{Recon}$$

f. Take gradient w.r.t. W to update all learnable params W.

Store best model parameters according to validation

Transform :

a. Compute log map $L_{i,a,e} = \log(C C_{i,a,e})$ (for test)
b. Compute $C'_{i,a,e} = E^*(U^*(L_{i,a,e}))$
c. Use $C'_{i,a,e,out} = C'_{i,a,e}$ as the pre-aligned Covariances with preprocessing done.

Output : Trained SPD parameters (W) & $C_{i,a,e,out}$.

Observation :

RA + TS-SVM-RBF : 54.98 %	RiFU + TS-SVM-RBF : 55.17 %
RA + TSLR : 52.89 %	RiFU + TSLR : 54.78 %
RA + MDM : 52.43 %	RiFU + MDM : 52.51 %
RA + TSA-LDA : 53.51 %	RiFU + TSA-LDA : 54.24 %

CLASSIFIERS

RiFUNetClassifier :

1. Input : Covariances $C_{i,a,e}$, $\hat{y}_{act}$, $\hat{y}_{sub}$.
2. Riemann Alignment : $C'_{i,a,e} = C_{ref}^{-1/2} C_{i,a,e} C_{ref}^{-1/2}$.
3. We discussed SPD U-Net above using congruence transform

$$\phi_{SPD} = W^T C_{i,a,e} W \Rightarrow W \in \mathbb{R}^{c \times d} \begin{cases} d < c : \text{encoder} \\ d > c : \text{decoder} \end{cases}$$

Forward step through RiFUNet :

(i) Compute $C_{out,i,a,e} = E(UCC_{i,a,e})$ using SPD U-Net.
(Congruence transform + log merges in residual connections)

4. Tangent Space Projection :
   (a) Once we get Cout , compute logmap $L_i = \log C_{Cout,i,a,e})$
   (b) $vec_\Delta(L_i) = \Sigma_i$
5. Initiate action head
5. Compute oss components (like before)
   (a) within action W(A)
   (b) between action B(A)
   (c) within subject W(S)
   (d) predict action using action head & compute L_{CE}^{act}.
   (e) reconstruction loss: R = Recon ($C_{i,a,e}$, C_{out})
6. Compute Loss :

$$L = \lambda_f(\lambda_{within_{act}} W(A) - \lambda_{bet_{act}} B(A) - \lambda_{within_{sub}} W(S)) + \lambda_{CE} L_{CE}^{act} + \lambda_{recon} R$$

7. Backpropagate & update learning parameters. RiFUNet , action head & subject head
8. Output : Classifier (Model) , feature extractor (Σ_i)
   & prediction $\hat{y}_{pred,act}$

SPDNetClassifier : Deep Congruence Networks

1. Input : Covariance $C_{i,a,e}$, $\hat{y}_{act}$, $\hat{y}_{sub}$
2. Riemann Alignment : $C'_{i,a,e} = C_{ref}^{-1/2} C_{i,a,e} C_{ref}^{-1/2}$
3. SPD-Net forward pass :
   for each layer l : $C^{(l+1)} = W_l^T C^{(l)} W$.
   obtain final SPD output $C_{i,a,e}^{out}$
4. Tangent-space feature extraction :
   $L_{i,a,e} = \log(C C_{i,a,e}^{out})$ $\Sigma_i = vec_\Delta(L_{i,a,e})$
5. Compute fisher action terms : $\mu = \frac{1}{n_a}\sum \mu_a$
   (a) compute action means μ_a & global mean μ
   (b) compute within$_{act}$ W(a) & between$_{act}$ B(a)
6. Compute Subject Alignment ratios :
   (a) Compute subject mean v_s & global mean v
   (b) Compute $S = \lambda_{between} B(s) - \lambda_{within} W(s)$
7. Compute Cross Entropy Loss
   Compute $\hat{y}_i^{act}$ using action head & compute L_{CE}.
8. Compute Loss = $\lambda_{CE} L_{CE} + \lambda_{ish}(\lambda_{within_{act}^i} W(a) - \lambda_{bet_{act}} B(a)) + \lambda_{subj} S$
9. Backpropagate & update Weights & classifier
10. Output. trained model classifier => feature extractor Σ_i &.
    prediction $\hat{y}_{pred,act}$.

APPENDIX

APPENDIX

(i) Symmetry of geodesic distance

$$d(C_1, C_2) = \| \log(C_1^{-1/2} C_2 C_1^{-1/2}) \|_F$$

$$d(C_2, C_1) = \| \log(C_2^{-1/2} C_1 C_2^{-1/2}) \|_F$$

Let $X = C_1^{-1/2} C_2 C_1^{-1/2}$

$(AB)^{-1} = B^{-1} A^{-1}$

$$X^{-1} = (C_1^{-1/2} C_2 C_1^{-1/2})^{-1}$$
$$= (C_2 C_1^{-1/2})^{-1} (C_1^{-1/2})^{-1}$$
$$= (C_1^{-1/2})^{-1} C_2^{-1} (C_1^{-1/2})^{-1}$$
$$= C_1^{1/2} C_2^{-1} C_1^{1/2}$$
$$X^{-1} = C_1^{1/2} (C_2^{-1} C_1) C_1^{-1/2}$$
$$X = C_1^{1/2} (C_1^{-1} C_2) C_1^{-1/2}$$

$$\text{eig}(X^{-1}) = \text{eig}(C_2^{-1} C_1)$$
$$\text{eig}(X) = \text{eig}(C C_1^{-1} C_2)$$

Property: if $A = PAP^{-1}$, $\text{eig}(A) = \text{eig}(PAP^{-1})$

$$(C_2^{-1} C_1)^{-1} = C_1^{-1} C_2$$

$\therefore$ eig $(C_2^{-1} C_1)$ = reciprocal (eig $(C_1^{-1} C_2)$)

$$\| \log X \|_F = \left(\sum_i (\log \lambda_i)^2 \right)^{1/2}$$
$$\| \log X^{-1} \|_F = \left(\sum_i (\log \lambda_i^{-1})^2 \right)^{1/2} = \left(\sum_i (\log \lambda_i)^2 \right)^{1/2}$$
$$= \| \log X \|_F$$

$\therefore \| \log X^{-1} \| = \| \log X \|_F \Rightarrow d(C_1, C_2) = d(C_2, C_1)$

(ii) Euclidean Mean $= \underset{C}{\text{argmin}} \sum_{i,a} d_E^2(C, C_{i,a})$

$$\nabla_C \sum_i d_E^2(C_i, C_i) = 2 \sum_{i,a} (C - C_{i,a}) = 0$$

$$C_{E,ref} = \frac{1}{N_T} \sum_{i,a} C_{i,a}$$

(iii) Log-Euclidean Mean $= \log\left(\underset{C}{\text{argmin}} \sum_i d_{LE}^2(C, C_i) \right)$

$$\nabla_C \sum_i d_{LE}^2(C_i, C_i) = 2 \sum_{i,a} (\log C - \log C_{i,a})(C^{-1}) = 0$$

$$C_{LE,ref} = \log C = \frac{1}{N_T} \sum_{i,a} \log C_{i,a}$$

(iv) $C_e' = C_{R,ref,i}^{-1/2} C_e C_{R,ref,i}^{-1/2}$

Proof

$C_{R,ref,i}$ = SPD matrix. $C_{ref} = C_{ref}^T$

We know $C_{R,ref,i}$ is unique for each i

So to generalize across a $i \in \{1, 2, 3 \ldots 109\}$

$C_{R,ref,1} = C_{R,ref,2} \ldots\ldots = C_{R,ref,109} = I$

$C_{R,ref,i} = I$ (All eigenvectors (axes) => unity)

$W C_{R,ref,i} W^T = I$.

$W = W^T = C_{R,ref,i}^{-1/2}$

$C_{R,ref,i}^{-1/2} C_{R,ref,i} C_{R,ref,i}^{-1/2} = I$ for all i

$$C_{R,ref,i} = \underset{C}{\text{argmin}} \sum_e d_R^2(C, C_e)$$
$$= \underset{C}{\text{argmin}} \sum_e \| \log(C^{-1/2} C_e C^{-1/2}) \|_F = C_{ref}$$
$$= \underset{C}{\text{argmin}} \sum_e \| \log(C^{-1/2} C_{ref}^{-1/2} C_e C_{ref}^{-1/2} C^{-1/2}) \|_F$$
$$= \underset{C}{\text{argmin}} \sum_e \| \log((C_{ref} C)^{-1/2} C_e (C C_{ref})^{-1/2}) \|_F$$
$$= \underset{C}{\text{argmin}} \sum_e \| \log(((C_{ref} C)^{-1/2} C_e (C C_{ref}^{-1/2})) \|_F$$

CC_{ref}; $CC_{ref} = C_{ref} C \Rightarrow C_{ref} = I$ or $C = I$

=> We know at this condition $CC_{ref} = C_{ref}$.

$\therefore C = I. \Rightarrow C_{ref}' = I$

We shaped all covariances such that we $C_{ref}' = I$ (all eigenvectors are unit vectors)

(v) Signal whitening => linear transformation

$$C_{i,a,e}' = C_{R,ref,i}^{-1/2} C_{i,a,e} C_{R,ref,i}^{-1/2}$$
$$= C_{R,ref,i}^{-1/2} \frac{1}{N-1} X_{i,a,e} X_{i,a,e}^T C_{R,ref,i}^{-1/2}$$
$$= \frac{1}{N-1} C_{R,ref,i}^{-1/2} X_{i,a,e} X_{i,a,e}^T C_{R,ref,i}^{-1/2}$$
$$= \frac{1}{N-1} (C_{R,ref,i}^{-1/2} X_{i,a,e})(C_{R,ref,i}^{-1/2} X_{i,a,e})^T$$
$$= \frac{1}{N-1} X_{i,a,e}' X_{i,a,e}'^T$$

$$X_{i,a,e}' = C_{R,ref,i}^{-1/2} X_{i,a,e}$$

$C \times C$. $C \times T \Rightarrow C \times T$

So $U_{ref} \Lambda_{ref}^{-1/2} U_{ref}^T X_{i,a,e}$.

rotate back, transform, rotate signal => linear transform

We normalize the signal to the reference coordinates.

(vi) $C_K \omega = \lambda (C_t) \omega$

$C_t^{-1/2} C_K C_t^{-1/2} \omega = \lambda (C_t^{-1/2} C_t C_t) \omega$

$S_K \omega = \lambda \omega$.

$S_K = C_t^{-1/2} C_K C_t^{-1/2}$

Let eigendecomposition of $C_t = E \Lambda E^T$

$C_t^{-1/2} = E \Lambda^{-1/2} E^T$.

eigendecomposition of $S_K = U \Lambda U^T$; $U^T U = I$

$S_K = U \Lambda U^{-1} \Rightarrow U^{-1} S_K U = U^T S_K U = \Lambda$

$U^T C_t^{-1/2} C_K C_t^{-1/2} U = \Lambda$ [$C_t^{-1/2} = (C_t^{-1/2})^T$]

$(C_t^{-1/2} U)^T C_K (C_t^{-1/2} U) = \Lambda$ [diagonal].

$W^T C_K W = \Lambda$., $W^T C_t W = I$.

$W^T = W^{-1} C_t^{-1} \Rightarrow W^{-1} C_t^{-1} C_K W = \lambda$.

$\boxed{C_K W = \lambda C_t W}$ => this is verification

$\therefore W = C_t^{-1/2} U$ where U is eigenvector of whitened covariance

(vii) Assume C classes, class C has Gaussian distribution. $N(x_c ; \mu_c, \Sigma)$ with same Σ

Bayes theorem:

$P(y = c/x) \propto P(x/y = c) P(y = c)$

$P(y = c) = \pi_c$ (prior)

$P(y = c/x) \propto P(x/y = c) . \pi_c$ — (1)

$x/y = c \sim N(\mu_c, \Sigma)$

$P(x/y = c) = \frac{1}{\sqrt{2\pi\Sigma}} e^{-(x-\mu_c)\Sigma^{-1}(x-\mu_c)^T}$

From (1): $\log P(y = c/x) = \log (p(x/y = c)) + \log \pi_c$

$\log (p(y = c/x)) = \log \left(\frac{1}{\sqrt{2\pi\Sigma}} e^{-\frac{1}{2}(x-\mu_c)\Sigma^{-1}(x-\mu_c)^T} \right) + \log \pi_c$

$= -\frac{1}{2}(x-\mu_c)\Sigma^{-1}(x-\mu_c)^T - \frac{1}{2}\log 2\pi\Sigma + \log \pi_c$

$= -\frac{1}{2} x\Sigma^{-1}x^T + \frac{1}{2} x\Sigma^{-1}\mu_c^T + \frac{1}{2}\mu_c\Sigma^{-1}x^T - \frac{1}{2}\mu_c\Sigma^{-1}\mu_c^T - \frac{1}{2}\log 2\pi\Sigma + \log \pi_c$

$= -\frac{1}{2}[x\Sigma^{-1}x - 2x^T\Sigma^{-1}\mu_c + \mu_c\Sigma\mu_c^T + \log 2\pi\Sigma] + \log \pi_c$.

We need to maximize $p(y = c/x) \propto \log p(y = c/x)$

=> depends only on class c (changing factor)

$x\Sigma^{-1}x$, $\log 2\pi\Sigma$ are constant => independent of c.

maximize => $x^T\Sigma^{-1}\mu_c - \frac{1}{2}\mu_c\Sigma\mu_c^T + \log \pi_c$

$\boxed{c = \underset{c}{\operatorname{argmax}} \left(x^T\Sigma^{-1}\mu_c - \frac{1}{2}\mu_c\Sigma\mu_c^T + \log \pi_c \right)}$

Case LDA: $x^T . V_K = \Sigma_K^T$. Otherwise the μ_c => distribution of $\Sigma_K^T . N(\Sigma_K ; \mu_{CK}, \Sigma_K)$

$c = \underset{c}{\operatorname{argmax}} \left(\Sigma_K^T \Sigma_K^{-1} \mu_{CK} - \mu_{CK}\Sigma_K\mu_{CK}^T + \log \pi_c \right)$